Monte Carlo Methods

Introduction

- *Monte Carlo (MC) Methods*
  - do not assume complete knowledge of environment (unlike DP methods which assume a perfect environment model)
  - require experience
    - *sample sequences of states, actions, and rewards from interaction (on-line or simulated) with an environment*
  - on-line experience exposes agent to learning in an unknown environment whereas simulated experience does require a model, but the model needs only generate sample transitions and not the complete probability distribution like in DP.
- In *MC* methods, the *unknown value functions* are computed from *averaging sample returns*.

Set up

- We assume the environment is a finite MDP.
- Finite set of states: *S*
- Finite set of actions: *A(s)*
- Dynamics of environment given by:
  - a set of transition probabilities, and
  - $P^a_{s_i s_j} = P(s_{t+1} = s_j | s_t = s_i, a_t = a)$ unknown
  - the expected immediate reward
  - $R^a_{s_i s_j} = E(r_{t+1} | s_t = s_i, a_t = a, s_{t+1} = s_j)$ unknown
- for all $s_i \in S$, $s_j \in S^+$ and a $\in A(s)$.

MC Policy Evaluation

- We begin by considering Monte Carlo methods for learning the state-value function for a given policy.
- *Value of a state:* is the expected return— *expected cumulative future discounted reward -- starting from that state.*
- To estimate *state value* from *experience*, *returns observed after visits to that state should averaged*. This idea underlies all Monte Carlo methods.
- Each occurrence of state s in an episode is called a visit to s.
- *Every-visit MC method*: method of estimating state-value function i.e. $v_\pi(s)$ as the average of the returns following all visits to the state *s* in a set of episodes.
- *First-visit MC method*: method of estimating $V^\pi(s)$ as the average of the returns following first visits to the state *s* in a set of episodes.

First-visit MC Method

- *Initialize:*
- π: *the policy to be evaluated*
- *V: an arbitrary state-value function*
- *Returns(s): an empty list, for all* $s \in S$
- *Repeat forever:*
  - *Generate an episode using* π
  - *For each state s appearing in the episode:*
    - $R \leftarrow$ *return following the first occurrence of s*
    - *Append R to Returns(s)*
    - $V(s) \leftarrow average(Returns(s))$

First-visit MC Method

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

DP vs Monte Carlo

DP Learning

- DP methods require an explicit system model to describe every system transition and the associated rewards. DP needs a MDP model with complete knowledge of dynamics
- It can be applied to episodic and continuing events.
- The learning process of DP needs to sweep the entire state space, and the computational requirement grows exponentially with the number of states
- we compute value functions from knowledge of the MDP.
- DP methods are bootstrap.

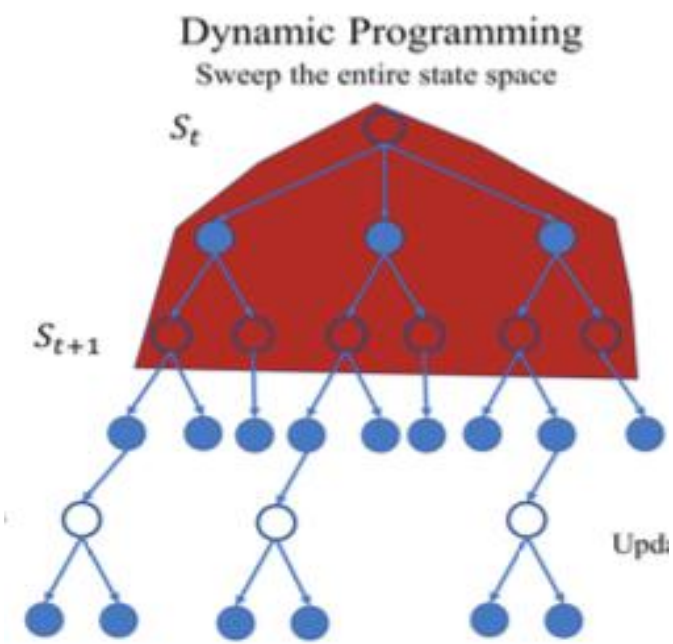
Monte Carlo Learning

- Model Free learning. Monte Carlo methods require only experience – online or simulated
- It assumes the episodic events where each episodes terminates whatever be the action selection.
- It does not require to learn by sweeping through all state space. On the contrary, it start from state s until final episode state and calculate the average returns.
- we learn value functions from sample returns with the MDP
- In Monte Carlo methods , the estimates for each state are independent. The estimate for one state does not build upon the estimate of any other state, as is the case in DP. In other words, Monte Carlo methods do not bootstrap

DP vs Monte Carlo

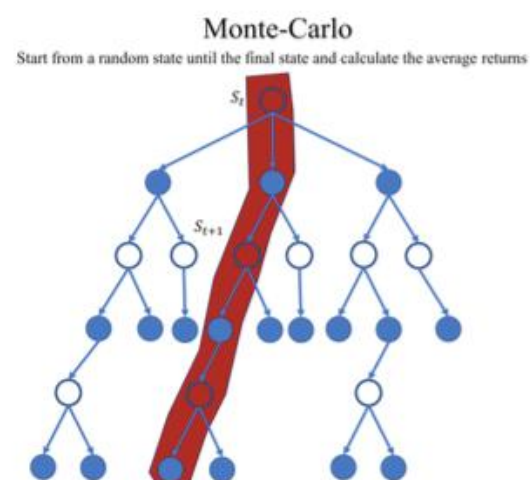
DP Learning

- The agent uses the current policy π to predict the value function through an iterative approach.
- computational expense of estimating the value of a single state is higher as number of states grows exponentially.



Monte Carlo Learning

- The idea behind the Monte-Carlo method is sampling and calculating average returns. Under a policy, the agent randomly selects a state, takes actions, gets rewards, and calculates returns. Afterward, the value functions are updated using the average returns.
- computational expense of estimating the value of a single state requires the value of only one or a subset of states. One can generate many sample episodes starting from the states of interest, averaging returns from only these states, ignoring all others



MC Estimation of Action Values

- For environments without a model(MC), state values alone are not sufficient to determine a policy. So focus on action-value.
- For model with environment knowledge(DP), A *one-step look-ahead* operation is enough to find the action that leads to the best combination of reward and next state.
- We need to estimate the value of the state-action pair, $q_\pi(s,a)$, the expected return when starting in state *s*, taking action *a*.
- A state– action pair s, a is said to be visited in an episode if ever the state s is visited and action a is taken in it.
- The every-visit MC method estimates the value of a state–action pair as the average of the returns that have followed all the visits to it. The first-visit MC method averages the returns following the first time in each episode that the state was visited and the action was selected.

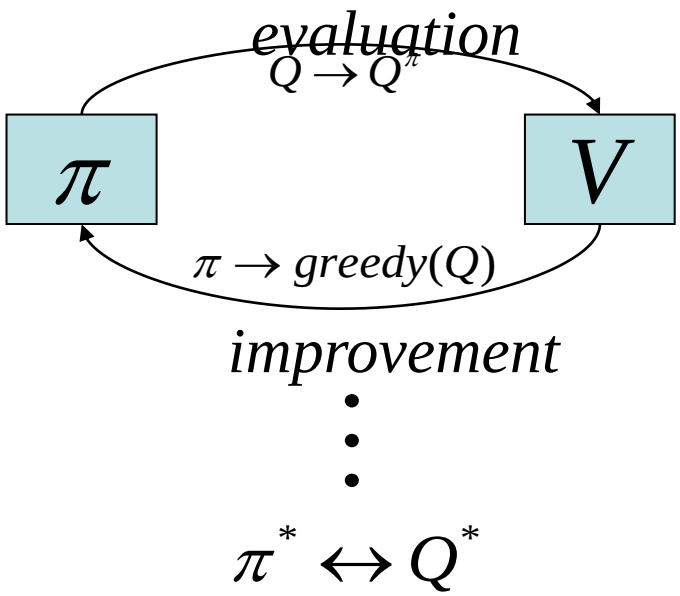
- Thus, one of our primary goals for Monte Carlo methods is to estimate $q^*(s, a)$ for policy evaluation.

MC Estimation of Action Values

- ***Problem***: many state-action pairs may never be visited.
  - We need to *maintain exploration.*
  - *Two assumptions* to solve the problem*:*
    - *Exploring starts: each state-action pair has a non-zero probability.*
    - *Infinite number of episodes: all state-action pairs are visited infinitely many times, called as exploring starts.*

Generalized Policy Iteration in MC Methods

In a Monte Carlo version of classical policy iteration, we perform alternating complete steps of policy evaluation and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function (GPI)



where π^* and Q^* denote optimal policy and action values, respectively.

Policy Iteration: MC Version

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \xrightarrow{I} \cdots \pi_{opt} \xrightarrow{E} Q^{\pi_{opt}}$$

In the above diagram denoting the policy iteration the transition *E* and *I* stand for the policy evaluation and policy improvement phases of the poliy iteration steps in respective order [1]. For any *Q*, the corresponding *greedy policy* selects an *action with* the *maximal Q-value* as in the following:

$$\pi(s) = \arg\max_a Q(s,a)$$

Removing the Second Assumption...

- To remove the assumption of *infinite number of episodes* we again use the same idea of *value iteration* (i.e., instead of many just a single iteration of policy evaluation between each policy improvement step) as we did during the discussion of dynamic programming.
- An extreme version is the *in-place value iteration* where at each iteration only one state is updated.

Removing the Second Assumption...

- For Monte Carlo policy iteration it is natural to alternate between evaluation and improvement on an episode-by-episode basis.
- After each episode, the observed returns are used for policy evaluation, and then the policy is improved at all the states visited in the episode.
- A complete simple algorithm along these lines, which we call Monte Carlo ES, for Monte Carlo with Exploring Starts, is given in next slide.

Monte Carlo ES Method

- *Initialize π(s) and Q(s,a) arbitrarily for all s∈S, a∈A(s):*
- *Returns(s,a) ← empty list*
- *Repeat forever:*
  - *Generate an episode using exploring starts and π*
  - *For each pair (s,a) appearing in the episode:*
    - *R ← return following the first occurrence of s,a*
    - *Append R to Returns(s,a)*
    - *Q(s,a) ← average(Returns(s,a))*
  - *For each s in the episode:*

$$\pi(s) \leftarrow \arg\max_a Q(s,a)$$

Removing the first Assumption...

- To remove the assumption of *exploring starts* the agent needs to continually select actions.

- Two types of MC control to select actions are:
  - *on-policy* MC control, and
  - *off-policy* MC control.

On-Policy MC Control

- In *on-policy* control, the policy we use to select actions is the same as the policy we use to estimate action values.
- Idea is that of GPI(Generalized Policy Iteration).
- Without ES, the policy moves to an ε-greedy one.

On-Policy MC Control Algorithm

- *Initialize for all s∈S, a∈A(s):*
- *Q(s,a) ← arbitrary*
- *Returns(s,a) ← empty list*
- *Repeat forever:*
  - *Generate an episode using π*
  - *For each pair (s,a) appearing in the episode:*
    - *R ← return following the first occurrence of s,a*
    - *Append R to Returns(s,a)*
    - *Q(s,a) ← average(Returns(s,a))*
  - *For each s in the episode:*

$$a^* \leftarrow \arg\max_a Q(s,a)$$

$$\pi(s,a) \leftarrow \begin{cases} 1-\varepsilon+\varepsilon/|A(s)| & if \quad a=a^* \\ \dfrac{\varepsilon}{|A(s)|} & if \quad a \neq a^* \end{cases}$$

On-Policy MC Control Algorithm

That any ε-greedy policy with respect to q_π is an improvement over any ε-soft policy π is assured by the policy improvement theorem. Let π' be the ε-greedy policy. The conditions of the policy improvement theorem apply because for any $s \in \mathcal{S}$:

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) q_\pi(s,a) \\ &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \;+\; (1-\varepsilon) \max_a q_\pi(s,a) \qquad (5.2) \\ &\geq \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \;+\; (1-\varepsilon) \sum_a \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1-\varepsilon} q_\pi(s,a) \end{aligned}$$

(the sum is a weighted average with nonnegative weights summing to 1, and as such it must be less than or equal to the largest number averaged)

$$\begin{aligned} &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \;-\; \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \;+\; \sum_a \pi(a|s) q_\pi(s,a) \\ &= v_\pi(s). \end{aligned}$$

Thus, by the policy improvement theorem, $\pi' \geq \pi$ (i.e., $v_{\pi'}(s) \geq v_\pi(s)$, for all $s \in \mathcal{S}$). We now prove that equality can hold only when both π' and π are optimal among the ε-soft policies, that is, when they are better than or equal to all other ε-soft policies.

Off-Policy MC Control

- In *off-policy* control, the policy we use to select actions (*behavior policy*), π_b, and the policy we use to estimate action values (*estimation policy*), π_e, are separated.
- The question here is, how the state value estimates will be correctly updated using the estimation policy to reflect the selections of the behavior policy.

Off-Policy MC Control Via Importance sampling

- **Behaviour Policy b**: The policy used to generate episodes, i.e., the policy the agent follows to explore the environment.
- **Target Policy π**:The policy we want to evaluate, i.e., the one whose value function we are interested in.
- **Importance sampling** is a statistical technique used to estimate the properties of a particular distribution while using samples generated from a different distribution. The key idea is to reweight the samples from one distribution (called the **behavioural distribution**) to make them representative of the **target distribution**. This allows for more efficient sampling, especially in situations where sampling directly from the target distribution is difficult or costly
- **Importance Sampling Ratio**: To adjust for the fact that episodes are generated by the behaviour policy instead of the target policy, we use importance sampling ratios to weight the returns. The importance sampling ratio for a sequence of actions is:
- $$p_t = \pi(a_t|s_t) \,/\, b(a_t|s_t)$$
- where $\pi(a_t|s_t)$ is the probability of selecting action a_t under the target policy π , and $b(a_t|s_t)$ is the probability under the behaviour policy.

Off-Policy MC Control Via Importance sampling

- First requirement to work this out is that actions taken in π_e are also taken in π_b. that is, we require that $\pi(a|s) > 0$ implies $b(a|s) > 0$. This is called the assumption of coverage. It follows from coverage that b must be stochastic in states where it is not identical to π $(\pi \neq b)$.

- Second, the rates of possibility of occurrence of any sequence between π_e and π_b starting from a specific visit to state s should be balanced.

- That is, consider i^{th} first visit to state s and the complete sequence of states and actions following that visit. We define $p_{i,e}(s)$ and $p_{i,b}(s)$ as the probability of occurrence of the sequence mentioned above given policies π_e and π_b, respectively. Further, let $R_i(s)$ denote the corresponding observed return from state s, and $T_i(s)$ be the time of termination of the ith episode involving state s. Then,

Off-Policy MC Control Via Importance sampling... (2)

- Given a starting state S_t, the probability of the subsequent state–action trajectory, A_t, S_{t+1}, A_{t+1}, . . . , S_T , occurring under any policy π is

$$p_i(s_t) = \prod_{k=t}^{T_i(s)-1} \pi(s_k, a_k) P_{s_k s_{k+1}}^{a_k}$$

- Assigning weights to each return by its relative probability of occurrence under π_e and π_b, $p_{i,e}(s)/p_{i,b}(s)$, the value estimate can be obtained by

$$V(s) = \frac{\sum_{i=1}^{n_s} \frac{p_{i,e}(s)}{p_{i,b}(s)} R_i(s)}{\sum_{i=1}^{n_s} \frac{p_{i,e}(s)}{p_{i,b}(s)}}$$

- where n_s is the number of returns observed from state s. This is weighted sampling.

Off-Policy MC Control Via Importance sampling... (3)

- Looking below at the ratio of the probability (**relative probability**) we see that it depends upon the policies and not at all on the environment's dynamics.

$$\frac{p_{i,e}(s_t)}{p_{i,b}(s_t)} = \frac{\prod_{k=t}^{T_i(s)-1} \pi_e(s_k, a_k) P_{s_k s_{k+1}}^{a_k}}{\prod_{k=t}^{T_i(s)-1} \pi_b(s_k, a_k) P_{s_k s_{k+1}}^{a_k}} = \frac{\prod_{k=t}^{T_i(s)-1} \pi_e(s_k, a_k)}{\prod_{k=t}^{T_i(s)-1} \pi_b(s_k, a_k)}$$

Every-visit MC algorithm for off-policy policy evaluation (Incremental)

- A complete every-visit MC algorithm for off-policy policy evaluation using weighted importance sampling is given below

Suppose we have a sequence of returns $G_1, G_2, \ldots, G_{n-1}$, all starting in the same state and each with a corresponding random weight W_i (e.g., $W_i = \rho_{t_i:T(t_i)-1}$). We wish to form the estimate

$$V_n \doteq \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}, \qquad n \geq 2, \tag{5.7}$$

and keep it up-to-date as we obtain a single additional return G_n. In addition to keeping track of V_n, we must maintain for each state the cumulative sum C_n of the weights given to the first n returns. The update rule for V_n is

$$V_{n+1} \doteq V_n + \frac{W_n}{C_n}\Big[G_n - V_n\Big], \qquad n \geq 1, \tag{5.8}$$

and

$$C_{n+1} \doteq C_n + W_{n+1},$$

Every-visit MC algorithm for off-policy policy evaluation (Incremental)

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$

Loop forever (for each episode):
  $b \leftarrow$ any policy with coverage of π
  Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
    $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Off-Policy MC Control Algorithm

Off-policy Monte Carlo control method, based on GPI and weighted importance sampling, for estimating π_* and q_*

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
- $Q(s,a) \in \mathbb{R}$ (arbitrarily)
- $C(s,a) \leftarrow 0$
- $\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
- $b \leftarrow$ any soft policy
- Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- $W \leftarrow 1$
- Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
  - $G \leftarrow \gamma G + R_{t+1}$
  - $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
  - $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\left[G - Q(S_t, A_t)\right]$
  - $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)
  - If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
  - $W \leftarrow W \frac{1}{b(A_t|S_t)}$

References

- [1] Sutton, R. S. and Barto A. G., "*Reinforcement Learning: An introduction*," MIT Press, 1998